

# Dawuro Platform — Frontend Architecture

Version 2.1 · 2 September 2026 · supersedes the GhanaMediaWatch frontend architecture (v1, 19 August)

## 0. What this document is

Dawuro has **two frontends and one shared rules package**. v1 of this document described only the mobile app, because the web console did not exist when it was written. This version covers both, plus how they are kept in agreement.

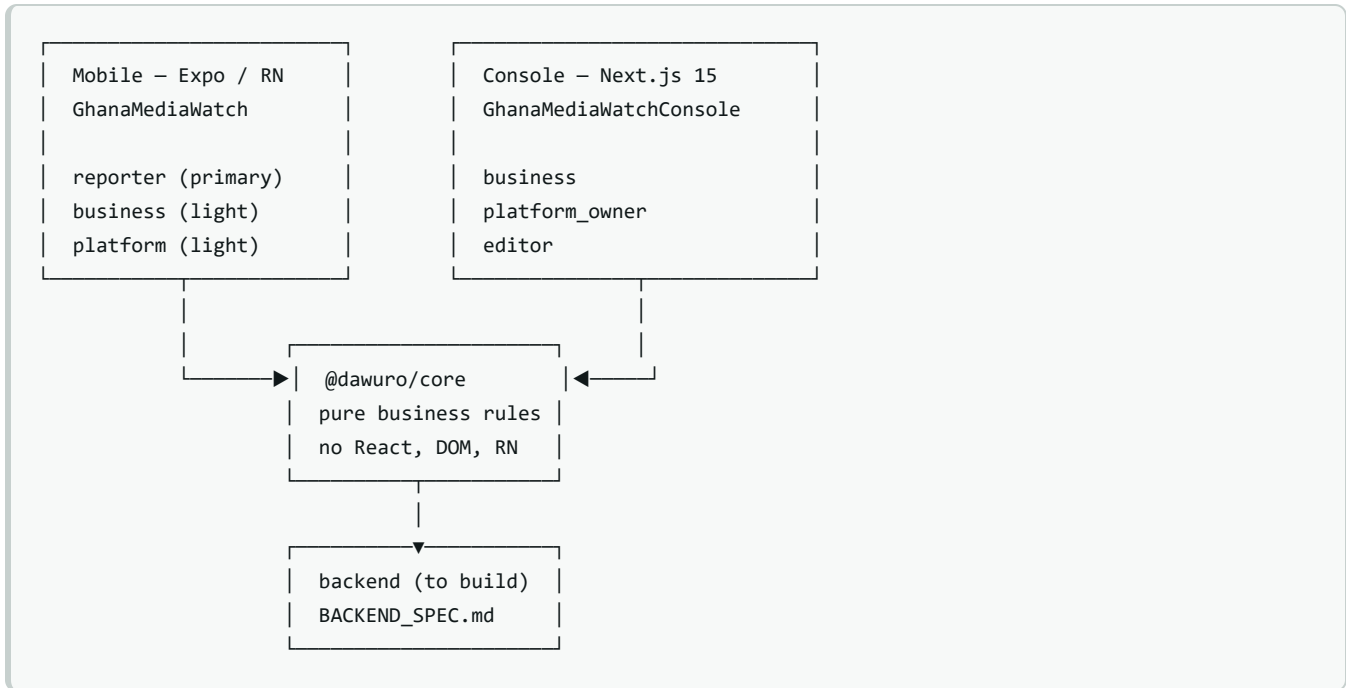
**Audience:** anyone who needs to understand the clients without working on them — most immediately the backend engineer, as the companion to `BACKEND_SPEC.md`.

**Status is stated plainly throughout.** v1 carried a phase table saying only Phase 1 was built; that has not been true for some time. Everything described here is built and running against fixtures unless a section says otherwise, and §12 is the honest inventory of what is not.

### What changed since v1

|              | V1 SAID                                 | ACTUALLY                                                                                 |
|--------------|-----------------------------------------|------------------------------------------------------------------------------------------|
| Products     | One (mobile)                            | <b>Two</b> — mobile app and web console                                                  |
| Expo SDK     | 57                                      | <b>54</b>                                                                                |
| React Native | 0.86.2                                  | <b>0.81.5</b>                                                                            |
| React        | 19.2.3                                  | <b>19.1.0</b>                                                                            |
| Expo Router  | 57.0.14                                 | <b>6.0.24</b>                                                                            |
| Camera       | <code>react-native-vision-camera</code> | <code>expo-camera</code> — vision-camera was never installed                             |
| Expo Go      | "will not work from Phase 4"            | <b>Works.</b> <code>npm start</code> runs <code>expo start --go</code>                   |
| Palette      | Warm paper <code>#FAF6F0</code>         | <b>Light Glass</b> , cool neutrals on <code>#F4F5FA</code> , accent <code>#5B3DF5</code> |
| Build state  | Phase 1 only                            | All phases built; 261 mobile tests, 370 core tests                                       |

# 1. The shape of the system



## Who uses which

**The reporter is mobile-only, permanently.** Camera, GPS gating and offline capture *are* the product, and none of them survive a browser. A reporter who signs into the console lands on `/no-console`, which explains why rather than showing an empty dashboard.

**The editor is console-only.** Editorial judgement means reading corroboration side by side; a phone is the wrong instrument.

**Business and platform\_owner have surfaces on both**, and this is the one piece of genuine duplication in the system — see §3.3.

## The division of labour with the backend

The split matters because several things a server might expect to own are already solved on the device.

**The clients own:** media capture, the GPS gate, local persistence of media and metadata, the outbox queue and its state machine, retry scheduling and backoff, background upload continuation, connectivity detection, Wi-Fi-only deferral, chunk resumption bookkeeping, and all rendering of verification state, assurance class and privacy flags.

**The server owns:** durable storage, the assurance classification itself, editorial decisions, the public feed, clustering, routing, authorisation, money, and abuse handling.

**Both own:** display-flag enforcement (§8) and the business rules in `@dawuro/core`. That is deliberate duplication, not redundancy — and in both cases **the server's answer is authoritative**. The client copy exists to make the interface honest and instant, never to secure anything.

## 2. Stack

### Mobile — GhanaMediaWatch

| CONCERN      | CHOICE                      | VERSION                      | WHY                                                                                                      |
|--------------|-----------------------------|------------------------------|----------------------------------------------------------------------------------------------------------|
| Framework    | Expo (managed)              | SDK <b>54</b>                | Declarative native config; Expo Go still works for demos                                                 |
| Runtime      | React Native                | <b>0.81.5</b> / React 19.1.0 | —                                                                                                        |
| Routing      | Expo Router, typed routes   | <b>6.0.24</b>                | File-based; typed routes catch broken links at compile time                                              |
| Styling      | NativeWind v4 + Tailwind v3 | 4.2.6                        | One styling language app-wide. NativeWind v5 is still preview                                            |
| Client state | Zustand                     | 5.0.15                       | Capture session and filters are small, imperative, and read from <b>non-React code</b> (the sync engine) |
| Server state | TanStack Query              | 5.101.4                      | Caching, pagination and offline persistence are the whole problem                                        |
| Local DB     | expo-sqlite + Drizzle       | 16.0.10 / 0.45.2             | The outbox is a real relational queue with a state machine                                               |
| Camera       | <b>expo-camera</b>          | 17.0.10                      | Bundled in Expo Go, which vision-camera is not — and that constraint decides the demo story              |
| Video        | expo-video                  | 3.0.16                       | Replaces the deprecated <code>expo-av</code> player                                                      |
| Audio        | expo-audio                  | 1.1.1                        | Voice reports (\$5.5)                                                                                    |
| Maps         | react-native-maps           | 1.20.1                       | —                                                                                                        |
| Lists        | @shopify/flash-list         | 2.0.2                        | The feed must hold 60 fps on mid-range Android                                                           |
| Forms        | react-hook-form + zod       | 7.85 / 4.4.3                 | One schema drives validation and the submit payload's types                                              |
| i18n         | i18next                     | 26.3.6                       | English only today, structured for a second locale from day one                                          |

## Console — GhanaMediaWatchConsole

| CONCERN      | CHOICE                | VERSION       | WHY                                                                                         |
|--------------|-----------------------|---------------|---------------------------------------------------------------------------------------------|
| Framework    | Next.js App Router    | <b>15.1.2</b> | Server Components keep fixture data off the wire; route groups give each role its own shell |
| Runtime      | React                 | 19.0.0        | —                                                                                           |
| Styling      | Tailwind v3           | 3.4.17        | Same token names as mobile (\$7)                                                            |
| Server state | TanStack Query        | 5.62.7        | —                                                                                           |
| Tables       | TanStack Table        | 8.20.6        | The routing desk and payout ledger are real data grids                                      |
| Client state | Zustand               | 5.0.2         | —                                                                                           |
| Validation   | zod                   | 3.24.1        | —                                                                                           |
| Auth         | <b>jose</b>           | 5.9.6         | Signs and verifies JWTs on the <b>edge runtime</b> , which <code>jsonwebtoken</code> cannot |
| Icons        | lucide-react          | 0.468.0       | —                                                                                           |
| Monorepo     | <b>npm workspaces</b> | —             | Not pnpm, not yarn                                                                          |

**TypeScript is strict in both**, with `noUncheckedIndexedAccess` and `noUnusedLocals`. `any` is an ESLint **error**, not a warning.

## 3. Repository layout

### 3.1 Two repositories

```
GhanaMediaWatch/                the Expo app
src/
  app/                          expo-router routes only – thin, they compose feature components
  features/                     auth business capture comments earnings feed incident
                                map org outbox platform profile settings surveys
  components/ui/                design-system primitives
  api/                          ApiClient interface, MockApiClient, HttpApiClient, fixtures
  db/                           drizzle schema, migrations, repositories
  services/                     location, permissions, device identity, sync, media
  stores/ hooks/ types/ lib/ i18n/
scripts/                        documentation pipeline (§10.3)
BACKEND_SPEC.md FRONTEND.md API_CONTRACT.md (superseded)

GhanaMediaWatchConsole/         npm workspaces monorepo
packages/core/                  @dawuro/core – pure rules, no React/RN/DOM/Next
apps/console/                   the Next.js app
```

Two rules hold the mobile tree together, unchanged since v1 and still worth stating:

1. **Route files contain no business logic.** They import a feature component and render it.
2. **Cross-feature imports go through `components/ui` , `services` , or `types` only.**  
`features/feed` never imports from `features/capture` .

### 3.2 The drift problem — read this

`@dawuro/core` lives in the console repo. The mobile app is a **separate repository** and cannot resolve an npm workspace across a filesystem boundary, so it carries **hand-synced copies** of the shared code:

| FILE                            | MOBILE COPY                      | CONSOLE ORIGINAL                               | CURRENTLY                                                                                |
|---------------------------------|----------------------------------|------------------------------------------------|------------------------------------------------------------------------------------------|
| <code>format.ts</code>          | <code>src/lib/</code>            | <code>packages/core/src/lib/</code>            | Identical but for import paths                                                           |
| <code>placeholder.ts</code>     | <code>src/lib/</code>            | <code>packages/core/src/lib/</code>            | Identical but for import paths                                                           |
| <code>context.ts</code>         | <code>src/types/</code>          | <code>packages/core/src/types/</code>          | Identical but for a comment                                                              |
| <code>dawuro.ts</code>          | <code>src/types/</code>          | <code>packages/core/src/types/</code>          | <b>67 lines apart</b> — the console knows an <code>editor</code> role the phone does not |
| Design tokens                   | <code>global.css</code>          | <code>apps/console/src/app/globals.css</code>  | All 25 identical                                                                         |
| <code>SUBSCRIPTION_PLANS</code> | <code>src/types/dawuro.ts</code> | <code>packages/core/src/types/dawuro.ts</code> | Prices agree today                                                                       |

**The prices agree today. That is the whole risk.** Nothing enforces it, and the first time a tier changes in one repo and not the other, the phone will quote a reporter one figure and the console will invoice another. Two fixes, either acceptable:

- publish `@dawuro/core` to a private registry and depend on a version, or
- merge both apps into the existing workspaces monorepo.

Doing this **before the backend lands is cheap; after is not.**

### 3.3 The business and platform duplication

The mobile app has `business/` and `platform/` route trees built before the console existed, and the console now has fuller versions of the same surfaces. Both are live.

This is not accidental scope creep worth deleting on sight — a field officer approving a membership request from a phone is a real use — but it is **two implementations of one set of rules**, and it should be a deliberate decision rather than an accumulation. The recommendation is to keep mobile business/platform deliberately thin (inbox, approve, assign) and let the console own everything analytical.

## 4. The shared core

`packages/core` holds every rule that must produce the same answer in both clients and on the server: routing, assignment, commission, billing, permissions, the assurance and verification model, the editorial gate, onboarding, survey validation, clustering and the response/SLA log.

**It imports nothing from React, React Native, Next, the DOM, or Node.** Anything touching a camera, filesystem, network or screen does not belong in it. That constraint is what makes the package testable in isolation and reusable on a server.

~3,600 lines, **370 tests across 17 suites**, all passing. One of those suites, `specVectors.test.ts`, asserts the exact numbers printed in `BACKEND_SPEC.md` §15 — so the spec fails CI if the logic moves underneath it.

The full contents are documented in `BACKEND_SPEC.md` §3–§12 rather than repeated here.

---

## 5. Mobile architecture

### 5.0 The home screen is a newsroom feed

Worth stating first, because it is the screen the whole app is judged on and it changed shape after v2.0.

It is a **news list**, not a social card feed: a thumbnail on the left, headline right, and a meta line of category · time · comment count. Above it sits a fixed black masthead carrying the GNA lockup and a strip of **desks** — Ghana, Africa, World, Business, Politics, Sport. A reader navigates by desk, the way they would pick up a section of a paper.

Three consequences for the backend:

1. **section is required on every published incident** (`BACKEND_SPEC.md` §3.8). A report with no desk does not appear in the feed at all — silently, with no error and no empty state. It is the failure mode most likely to be read as data loss.
2. **A desk is not a category.** `category` is filed by the reporter and drives routing, commission and the editorial queue; `section` is chosen by an editor at release and drives nothing but placement. Never derive one from the other.
3. `GET /incidents` needs a repeatable `section` filter. The client currently filters locally over fixtures — a stand-in, not a design.

There is also a **slides mode** — a full-screen, auto-advancing reader for people who would rather be shown the news than scroll it. It reads the same feed payload; no separate endpoint.

#### 5.0.1 One fixed appearance

The app has **one theme**: a black masthead over a white page, with no way to change it. A dark/light switch was built and removed. It gave one question — what colour is this app — three sources of truth that disagreed: a value stored on the device, the reader's OS setting, and the app's own default. The app would come up dark for someone who had never asked for dark, and each fix moved the failure to a different one of the three.

Nothing here concerns the backend except as a warning about the shape of that bug: it produced no error, and the visible symptom pointed at the palette, which was the one part that was correct.

### 5.1 The API layer

Every screen is built against a single swappable interface, `ApiClient`, with two implementations chosen by one environment variable:

```

EXPO_PUBLIC_API_MODE=mock    → MockApiClient    seeded fixtures, artificial latency,
                                injectable failure rate, no backend needed
EXPO_PUBLIC_API_MODE=http    → HttpApiClient    real calls to EXPO_PUBLIC_API_URL

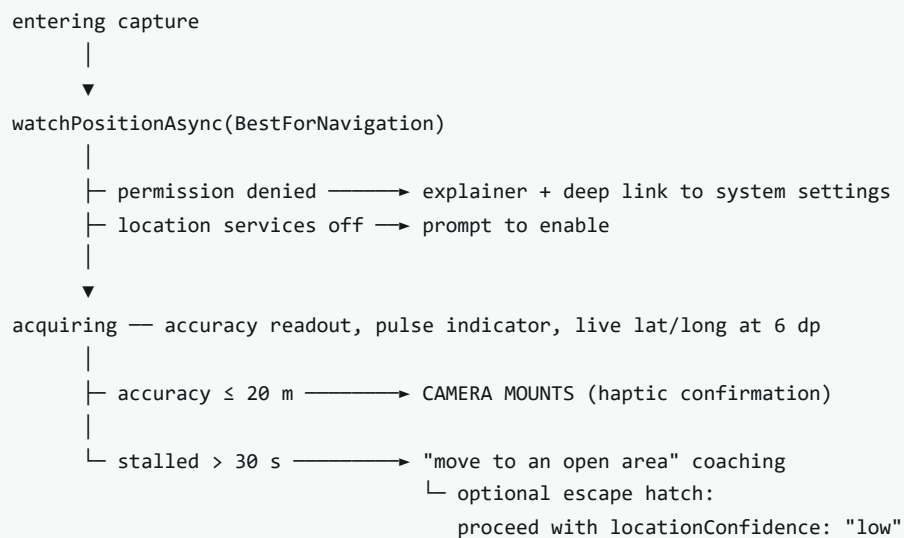
```

This is why frontend work never blocked on the backend, and why the contract was exercised end to end before a server existed.

**Consequence for the backend:** the client never constructs a request inline in a component. If your implementation must differ from the spec, the change lands in exactly one file.

## 5.2 The GPS gate

The defining feature. **The camera preview does not mount until a location fix meets the accuracy threshold.** A hard gate, not a warning.



| CONSTANT                                 | VALUE | RATIONALE                                                                                                                                                     |
|------------------------------------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>GPS_ACCURACY_THRESHOLD_M</code>    | 20    | About the best a consumer phone achieves outdoors in a few seconds. Tighter feels broken in urban canyons; looser is useless for sending someone to the scene |
| <code>GPS_STALL_TIMEOUT_MS</code>        | 30000 | When coaching and the escape hatch appear                                                                                                                     |
| <code>GPS_ABSOLUTE_MAX_ACCURACY_M</code> | 150   | Beyond this, not offered even via the escape hatch                                                                                                            |
| <code>MAX_VIDEO_DURATION_S</code>        | 60    | Enforced client-side with a ring countdown                                                                                                                    |

**The fix is locked at the shutter** and never re-read afterwards. A reporter who walks away before submitting must not have their report stamped with wherever they ended up.

**Tamper signals captured alongside it:** `accuracyM`, `altitude`, `heading`, `speed`, `capturedAtIso` (device clock), `capturedAtUtcOffsetMinutes`, `deviceUptimeMs`, and `isMocked` on Android. The reduced-accuracy escape hatch **flags** the report — it never silently downgrades it, and the flag is visible in the interface as well as in the payload.

All of these feed the server's `CaptureFacts` (`BACKEND_SPEC.md` §4.1). **The device does not classify itself.**

## 5.3 Offline-first sync

The network is treated as absent by default. Capture always succeeds locally.

On capture the client writes media to `documentDirectory/incidents/` — never the cache directory, which the OS can evict — and inserts a row in the local `incidents` table. Only when both succeed does the interface report success.

```
draft → queued → uploading → uploaded
                        |
                        └→ failed → (retry) → queued

any state → cancelled (terminal, user-initiated)
```

Persisted per row: `attemptCount`, `lastError`, `lastErrorCode`, `nextRetryAt`, `serverId`, `bytesUploaded`.

| ATTEMPT | DELAY              |
|---------|--------------------|
| 1       | 2 s                |
| 2       | 8 s                |
| 3       | 30 s               |
| 4       | 2 min              |
| 5       | 10 min             |
| 6+      | 1 h (cap, repeats) |

Plus **±25 % jitter**, so a thousand phones do not retry in unison when a cell tower returns. `Retry-After` on a `429` overrides the schedule entirely.

**Drain rules.** Subscribes to `NetInfo` and drains when `isInternetReachable === true` — not merely `isConnected`, which is true on a captive-portal Wi-Fi that goes nowhere. Oldest first, one at a time. A background task via `expo-task-manager` continues uploads when backgrounded; **iOS background execution is opportunistic and not guaranteed**, so the app also resumes draining on foreground. A Wi-Fi-only toggle defers *video* to Wi-Fi while photos still go over mobile data, and the outbox shows what is waiting and why.

**Deletion.** The local media file is removed **only after the server confirms completion**. The metadata row is kept, with `serverId`, so the user retains a personal history.

This is why media retention is an open question in the spec (§16). If the server expires footage the client has already deleted locally, the user's history points at nothing.

## 5.4 Capture modes

Four on the shutter: **photo**, **video**, **audio**, **live**.

Audio is a first-class report type and takes the same GPS lock as everything else — describing an incident from somewhere safe carries none of the risk of filming it, and it is the only mode that works in the dark or in a crowd. It is priced above a photo and below a video.

**Live is present but not implemented.** Selecting it opens a sheet that explains what it will do and offers to switch to video, rather than arming a shutter that does nothing. It is on the switch so the layout is settled before the feature arrives.

## 5.5 The capture stamp

Provenance is burned onto the frame rather than captioned beside it, because this footage is meant to travel and the claim has to travel with it: the report reference, the capture time at the precision the reporter allowed, and the location where permitted.

The same component renders on the feed card, the detail view and the console's media frame, so a still shared into WhatsApp carries the same reference someone can type into the public verification page.

## 5.6 Sharing

**One rule, and it lives in `src/lib/share.ts` precisely so it cannot be quietly broken at a call site: share the report, never the media file.**

A playback URL points at footage that may be licensed, restricted or pending review, and once it is pasted into a group chat none of those controls apply. The report page carries the context and can be taken down; a raw file cannot. The link is built from `reportId` — the reference stamped on the footage — so someone sent a clip and someone sent a link can tell they are looking at the same report.

---

# 6. Console architecture

## 6.0 Where a report reaches the public

The console is where a report becomes visible, so this is the part of it the backend most needs to understand.

`/published` is the business-side release surface. A business licenses a report privately, then decides separately whether the public sees it — releasing is a decision, not a consequence of paying. Releasing does two things at once:

- Credits the report to the organisation. This is the `publisher.kind: 'organisation'` variant ( `BACKEND_SPEC.md` §3.6). The reporter is still the author and still earns the commission; the organisation is only the publisher.
- **Sets the desk.** The picker is part of the release rather than a setting elsewhere, because a report released without a desk does not appear on the phone at all.

`/editorial` is the workbench where verification is decided. Note carefully that today `vettingState` is *derived* from `VerificationState` rather than stored: `vettingStateFor()` returns `published` the moment a report reaches `verified_high_confidence` or `verified_in_part`. So "is this true?" and "should the public see this?" are currently the same act, and partly-verified material publishes automatically.

**Treat that as a known design issue, not as the intended contract.** If the server is to enforce "nothing appears without approval" — which is the product rule — publication needs to be a stored decision with its own permission, separate from the verification judgement. See §12.

## 6.1 Route groups

Two — three, counting editorial — each with its own layout and sidebar. Route groups are a filesystem convention and **do not appear in the URL**: `(business)/inbox` serves `/inbox`.

```
app/  
  (business)/    inbox map published surveys team account onboarding  
  (platform)/    platform | routing | approvals | businesses | payouts  
  (editorial)/    editorial | decided  
  login register join invite/[token] verify verify/[reportId] no-console
```

`src/middleware.ts` gates by URL prefix, so **the route groups and the middleware prefix lists must be kept in step by hand**. Adding a page under `(business)/` without adding its prefix to `BUSINESS_PREFIXES` leaves it reachable by any signed-in account.

## 6.2 Auth

A signed JWT in an **httpOnly cookie**, so browser JavaScript can never read it.

Token primitives live in `src/lib/token.ts` with **no Node dependency**, because middleware runs on the edge runtime and must verify sessions with exactly the same code the server uses. `src/lib/session.ts` is the `server-only` half that reads cookies via `next/headers`. Splitting them is what lets middleware and Server Components reach identical conclusions about who someone is.

`jose` rather than `jsonwebtoken` for the same reason: it runs on the edge.

When the backend lands, **its access token is stored inside the session server-side** and attached by route handlers under `src/app/api/`. The browser must never see it.

## 6.3 Two rules that fail at runtime, not at build

**Never pass a component function across the RSC boundary.** Server layouts pass icon *names*; `src/components/shell/icons.ts` resolves them client-side. Passing the component itself fails with an opaque digest in production, not a type error in your editor.

**Never silently fall back to fixtures when the API is unreachable.** An outage must look like an outage. A console that quietly serves stale seed data during an incident is worse than one that says it is down.

## 6.4 Server Components by default

Pages are thin server routes that load data and render a client `*Workspace` component.

`(platform)/payouts/page.tsx` is sixteen lines: import the fixtures, render `<PayoutsWorkspace>`. This keeps fixture volume off the wire and puts interactivity in exactly the components that need it.

---

# 7. Design system — "Light Glass"

**All 25 design tokens are byte-identical across both repositories today.** They are hand-synced (\$3.2), which is the risk, but the system itself is genuinely one system.

Tokens live as CSS custom properties — `global.css` on mobile, `apps/console/src/app/globals.css` on the console — surfaced as Tailwind utilities with `<alpha-value>` support. Components never hardcode a hex; the single mirror is `src/lib/theme.ts`, for the few React Native APIs that take colours as props rather than styles.

| TOKEN                             | VALUE                    |                                   |
|-----------------------------------|--------------------------|-----------------------------------|
| <code>--color-canvas</code>       | <code>244 245 250</code> | <code>#F4F5FA</code> — the ground |
| <code>--color-canvas-soft</code>  | <code>255 255 255</code> | Cards and raised surfaces         |
| <code>--color-canvas-raise</code> | <code>233 236 244</code> | Wells and insets                  |
| <code>--color-text-primary</code> | <code>11 12 20</code>    | <code>#0B0C14</code>              |
| <code>--color-accent</code>       | <code>91 61 245</code>   | <code>#5B3DF5</code>              |
| <code>--color-accent-alt</code>   | <code>37 99 235</code>   | <code>#2563EB</code>              |

Channels are stored as space-separated RGB so Tailwind can compose alpha ( `bg-accent/25` ) without a second token per opacity.

**Light by default, and light is not a default — it is a decision.** Intraocular light scatter roughly doubles between age 20 and 70, so white-on-black haloes badly for an older reader. Dark mode exists, softened at both ends, and media surfaces stay dark in either scheme because a light frame wrecks the footage.

**Category colour is always a secondary cue.** Twenty-three hues cannot be reliably distinguished, particularly by an older eye, so every chip carries a text label and every map pin a distinct icon. Hue signals the *family* of incident; the label says which one.

Colour choices are asserted in `src/lib/__tests__/palette.test.ts`, so a later "soften that grey" fails CI rather than landing quietly.

---

## 8. Privacy enforcement

Three booleans per report: `showLocation`, `showDate`, `showTime`.

**Each client enforces these independently of the server**, with tests asserting a hidden value never reaches rendered output. The server enforcement in `BACKEND_SPEC.md` §3.5 is the other half.

Both layers are required, and the reason is worth stating plainly: **client-side hiding alone** means true coordinates travel to every phone that scrolls past the report and sit in an inspectable cache. **Server-side alone** means one client bug leaks them.

The review screen shows a **live public preview** — exactly what a stranger will see — so the reporter is never guessing.

**Anonymous mode** is a prominent toggle with an honest explainer. The copy states that anonymous means the public and other users see no profile, **but a device identifier is still attached** for abuse prevention. The client deliberately does not overstate the guarantee, and `deviceId` is never rendered in public UI on any screen.

## Device identity

A **UUID v4** generated on first launch and persisted in `expo-secure-store` is the primary identifier, sent as `X-Device-Id`. `Application.getAndroidId()` / `getIosIdForVendorAsync()` supplement it as a weak correlation signal only — **both platform IDs reset on uninstall or factory reset and are not stable identifiers**.

---

## 9. Quality bar

- **Accessibility** — `accessibilityLabel` and `accessibilityRole` on every interactive element; toggles carry `accessibilityState` so a filled heart is announced, not merely coloured; the capture flow announces accuracy and gate state to screen readers; reduced motion respected; system font scaling to 2× without breaking layout.
- **i18n** — every user-facing string goes through i18next. An automated sweep checks that every `t('...')` key resolves, so a missing translation fails rather than rendering a raw key.
- **Every list has three designed states** — skeleton, empty, error. A bare spinner is never an acceptable empty state.
- **Error handling** — a root error boundary plus per-screen boundaries, typed errors from the API layer, and toasts that say what happened and what to do next. **A raw stack trace is never shown to a user.**
- **No dead controls.** A button that does nothing is treated as a defect; a feature that does not exist yet explains itself (§5.4) rather than failing silently.

## Testing

|                           | SUITES | TESTS |
|---------------------------|--------|-------|
| Mobile                    | 23     | 261   |
| <code>@dawuro/core</code> | 17     | 370   |
| Console app               | 2      | 8     |

Priority targets are the pure logic — the sync state machine, the GPS gate, backoff, commission, routing, assignment, the editorial gate, permissions and the palette. Business rules are tested directly against fixtures rather than through a rendered screen, which is why they can be trusted to match the server.

CI gate, both repos: **typecheck, lint at zero warnings, tests.**

---

## 10. Running it

### 10.1 Mobile

**Expo Go works.** This changed since v1, and it changes the demo story — the app can be put on any phone by scanning a QR code, with no build and no cable.

```

npm install
npm start          # expo start --go, then scan the QR
npm run start:clear # same, clearing the Metro cache
npm run start:tunnel # when the phone is not on the same network

```

The constraint that buys this is **every native module must be one Expo Go already bundles** (§11.1). A development build is still available via `npm run start:dev-client` and is required if that constraint is ever broken.

```

npx expo prebuild --platform android
npm run android          # first build 10-20 min; later builds incremental

```

iOS requires a Mac. Android minSdk 26; iOS deployment target per Expo SDK 54.

| SCRIPT                         | DOES                                               |
|--------------------------------|----------------------------------------------------|
| <code>npm run typecheck</code> | <code>tsc --noEmit</code>                          |
| <code>npm run lint</code>      | ESLint, <b>zero warnings tolerated</b>             |
| <code>npm test</code>          | Jest                                               |
| <code>npm run docs:spec</code> | Regenerates the specification HTML and PDF (§10.3) |

## 10.2 Console

```

npm install          # from the repo root – npm workspaces
npm run dev          # console on :3000
npm run verify       # typecheck + lint + test, all workspaces
npm test             # @dawuro/core only

```

`apps/console/.env.local` needs `SESSION_SECRET` (32+ characters). See `.env.example`.

**Do not run `next build` while a dev server is live on the same directory.** The two compete over `.next` and leave a corrupted module graph whose symptom is every route returning 500 while typecheck and lint both pass. Recovery is `rm -rf .next` and a restart.

## 10.3 Documentation pipeline

Both PDFs are generated from markdown, so the prose has one source and never drifts from its rendering:

```

BACKEND_SPEC.md  ┌──┐
                  │  │
FRONTEND.md      └─┴─┐ scripts/render-spec.js → HTML → scripts/print-pdf.js → PDF

```

`print-pdf.js` drives whichever Chrome or Edge is already installed rather than pulling in Puppeteer, which would download a second Chromium into a repository with no other use for one. Set `CHROME_PATH` to override.

---

## 11. Known constraints

1. **Expo Go bundles a fixed set of native modules.** Anything outside that set means the app will not load in Go at all — not a degraded experience, a blank failure. This is why capture uses `expo-camera` rather than `react-native-vision-camera`, and it is a real constraint on every future native dependency. Check the SDK 54 bundled list before adding one.
2. **`react-dom` is pinned to 19.1.0 via `overrides`.** `expo-router` → `@expo/ui` → `vaul` floats it past Expo's pinned `react` and breaks npm peer resolution.
3. **React Native cannot synthesise font weights from one custom family on Android.** Each Inter weight is registered as its own family: `font-sans`, `font-sans-medium`, `font-sans-semibold`, `font-sans-bold`, `font-display`.
4. **Emulators report a perfect, mocked GPS fix** — precisely the branch the gate exists to detect. The capture flow needs a physical device to test meaningfully.
5. **Apple Maps has no routing coverage in Ghana.** Directions open Google Maps unconditionally — the app, then the web fallback. `LSApplicationQueriesSchemes` in `app.json` carries `comgooglemaps` so the app-scheme check works on iOS. Handing a Ghanaian user Apple Maps produces "Directions Not Available".
6. **Expo tunnel mode depends on `exp.direct` resolving and `connect.ngrok-agent.com` being reachable.** Some networks block both, and the failure looks like a hung startup rather than a network error. LAN mode is the fallback.
7. **Route groups and middleware prefixes are coupled by hand** (§6.1).
8. **`packages/core` cannot be resolved from the mobile repo** (§3.2). This is the one to fix.

---

## 12. What is built, and what is not

### Built and running against fixtures

**Mobile** — capture with the GPS gate and four modes, the offline outbox and sync engine, resumable chunked upload, the feed with comments and reactions, map and clustering, incident detail with in-app directions, earnings and the commission ledger, surveys, auth and onboarding, and light business and platform tiers.

**Console** — business inbox, routing desk, editorial workbench and decided log, institutional onboarding with per-step review, platform approvals, payouts, organisations, published archive, team management, invites and affiliations, the public verification page, and both role shells.

**Core** — every rule in `BACKEND_SPEC.md` §3–§12, with 370 tests.

### Not built

**The backend.** Nothing. That is `BACKEND_SPEC.md`.

**Specifically, the trust model has a hole until it exists.** `assurance` and `verification` render correctly and the state machine is enforced client-side, but the *facts* underneath — server-side integrity signing, server-time

comparison, device attestation, malware and duplicate screening, original preservation — are all server work ( `BACKEND_SPEC.md` §5.4). Until then, an assurance class is a display value with nothing behind it. **This is the single most important thing to build, because it is what the product claims to be.**

**Live streaming** is on the capture switch and explains itself (§5.4).

**Google sign-in** renders a button that is deliberately inert pending an OAuth client.

**Payouts** have a ledger and batches but no mobile-money integration.

### Known design issues the server must not inherit

These are not missing screens — they are places where the clients encode something the backend should decide differently. Each is verifiable in the code today.

**Publication is a side effect of verification.** `vettingStateFor()` derives `published` from `VerificationState`, so an editor cannot verify a report and hold it back, and `verified_in_part` — partly verified — goes live automatically. For a national news agency that is the wrong default.

**No capability governs the public feed.** The capability list has `view_editorial_queue`, `decide_verification` and `assign_editors`, and nothing about publishing. "Who may put a citizen's video in front of the country" is not expressible.

**The platform owner cannot approve anything.** `decide_verification` is held only by `editorial_lead` and `verification_editor`. `super_admin` does not have it, while its own description claims it holds every permission.

**The reporter's destination choice is collected and then dropped.** `DestinationPicker` is a real control on the review screen: the reporter chooses `public`, `marketplace`, `directed` or `both`, and picks named businesses for a directed submission. `submitCapture()` is then called without either value. So the single choice that decides whether a report earns a commission and who is allowed to see it never leaves the screen it was made on — and the server, when it exists, will receive nothing to route on.

**Human approval is the throughput ceiling.** If every report needs a person, the editorial queue is the product's capacity. Priority scoring exists — expedited class, then severity, then age, with age last so a stale observation cannot outrank a fresh emergency — but there is no time target or breach state for editorial the way `slaState` gives institutions one.

### Known rough edges

- Mobile and console **duplicate the business and platform tiers** (§3.3).
- The seeded fixtures are a scatter of unrelated reports rather than one coherent story that can be walked end to end.

---

## 13. What the backend engineer should take from this

- The clients are **offline-first and retry forever**. Assume duplicates; implement idempotency ( `BACKEND_SPEC.md` §2.6).
- The clients **already back off** on `429` and `5xx`. Return the right status and they behave. Return `500` for a validation error and every affected phone retries hourly,

indefinitely — the retryable/permanent split in §2.5 is not cosmetic.

- The mobile client **deletes local media on upload completion**. Your retention policy is not an internal detail; it changes what a user can see in their own history.
- The clients **enforce display flags too**, but that is defence in depth, **not permission to send suppressed values**.
- The clients **compute commission, routing and permissions locally** so the interface can be instant and honest. **Your answer is authoritative in every case**. Where the two disagree, the client is wrong and should be corrected — but a client that has to ask the server what a download costs before it can label a button is a client that shows a spinner where a price belongs.
- **The device never classifies its own assurance**. It sends facts; you decide (\$4.1, \$5.4).
- Everything is built against **one swappable interface** on mobile and thin route handlers on the console. Contract changes are cheap now and expensive once integration starts. **Raise them early**.